

OpenSkynet: A Self-Healing, Skill-Learning Framework for Persistent Browser Automation Agents

OpenSkynet Contributors

May 31, 2026

Abstract

Browser automation agents powered by large language models (LLMs) have shown promise in executing web-based tasks, but they face persistent challenges: brittle execution when web pages change, high token consumption from repeated trial-and-error, and the inability to retain and reuse successful strategies across sessions. We present OpenSkynet, an open-source framework that addresses these limitations through three integrated mechanisms: (1) a **self-healing skill executor** that automatically diagnoses and repairs failed automation steps by analyzing screenshots and DOM state via multimodal LLM reasoning; (2) a **skill learning pipeline** that extracts, refines, and merges reusable automation skills from observed trajectories; and (3) a **three-tier memory system** that progressively promotes procedural knowledge from working memory to persistent long-term storage, enabling cross-session skill improvement. OpenSkynet also introduces a multi-agent subagent architecture with permission-based tool isolation, parallel execution, and configurable browser backends. To evaluate self-healing and skill generalization under adversarial conditions, we propose the *Evil Skill Generalization Benchmark*, comprising 60 tasks across 20 templates with deliberately obfuscated web interfaces (hidden elements, injected content, multi-layer consent dialogs). OpenSkynet is released under the Business Source License 1.1.

1 Introduction

LLM-powered browser agents have emerged as a promising approach to automating complex web-based workflows. Systems such as Browser-Use [1], SeeAct [16], WebRL [10], and WebAgent [8] demonstrate that language models can navigate web pages, fill forms, extract data, and complete multi-step tasks with varying degrees of success. However, three fundamental limitations persist across existing systems:

1. **Brittle execution.** When web pages change their DOM structure, CSS selectors, or interaction patterns, pre-recorded automation scripts or skills break. Current systems require manual re-authoring or restart from scratch.
2. **No cross-session learning.** Each task execution is effectively stateless. Successful strategies discovered in one session are not retained, catalogued, or reused in future sessions, leading to redundant LLM calls and repeated failures.
3. **Single-agent limitations.** Complex multi-phase tasks (e.g., “research a competitor, extract pricing, update our database, and schedule a daily monitor”) exceed the effective context window of a single agent and benefit from parallel decomposition.

OpenSkynet is a modular, extensible framework designed to address each of these limitations. Its key contributions are:

- **A self-healing skill executor** that captures browser state (screenshot + DOM snapshot) at failure points, sends this multimodal context to an LLM for root cause diagnosis, and automatically patches the skill with corrected steps. Each healing cycle creates a versioned update, forming a continuous improvement loop without human intervention.

- **An automated skill learning pipeline** that observes agent trajectories, extracts reusable skills via LLM-based review with iterative refinement (up to 3 cycles), loads recent failure trajectories as negative examples, and merges similar skills to prevent duplication. Skills can also be learned from screen recordings via multimodal trace-to-skill conversion.
- **A three-tier memory architecture** with working, session, and long-term memory tiers. Entries are automatically promoted based on heuristic importance scoring and LLM-driven background review. Retrieval uses hybrid vector search (OpenAI → FastEmbed → TF-IDF fallback) combined with keyword overlap scoring.
- **A multi-agent subagent system** with 7 built-in specialized agents (browser, code, debug, explore, integrate, review, redteam) supporting permission-based tool isolation, browser sandboxing, parallel execution (up to 3 concurrent), and explicit nesting depth control to prevent infinite recursion.
- **The Evil Skill Generalization Benchmark**, a deliberately adversarial test suite of 60 tasks across 20 web application templates, each featuring 7 layers of anti-LLM obfuscation designed to stress-test skill generalization and self-healing capabilities.

The remainder of this paper is organized as follows. Section 2 describes the overall system architecture and agent loop. Section 3 details the skill learning and self-healing subsystems. Section 4 presents the tiered memory architecture. Section 5 covers the multi-agent subagent system. Section 6 introduces the Evil Skill Generalization Benchmark. Section 7 discusses related work, and Section 8 concludes with directions for future work.

2 System Architecture

2.1 Overview

OpenSkynet is organized into four conceptual layers, as illustrated in Figure 1: (1) the **orchestration layer**, containing the Manager Agent, Agent Loop, and subagent system; (2) the **skill layer**, responsible for learning, executing, healing, and auditing browser automation skills; (3) the **memory layer**, providing tiered persistent storage with hybrid retrieval; and (4) the **browser layer**, abstracting over multiple execution backends.

2.2 Manager Agent and Strategy Selection

The Manager Agent [1] is the entry point for all user requests. Upon receiving a task, it selects one of five strategies based on the task type and conversation context:

- **Conversational.** For informational queries that require no browser interaction, the manager responds directly.
- **Direct.** A straightforward browser automation task is passed directly to the Browser Agent.
- **Use-Skill.** If a learned skill matches the task description (via vector or keyword similarity), the existing skill is retrieved and executed.
- **Delegate.** Coding tasks are routed to the code subagent; complex tasks may be dispatched to specialized subagents.
- **Decompose.** Multi-phase tasks are decomposed into independent subtasks using beam-width planning (up to 5 subtasks across 2 beam candidates), scored by task coverage and parallelizability.

Strategy selection uses a multi-tier fast path: explicit URL navigation bypasses the LLM entirely; keyword-matching identifies coding tasks; an LLM classifier resolves ambiguous cases; and the full LLM planner is invoked only when necessary. This minimizes token consumption for simple tasks while preserving the model’s reasoning capability for complex ones.

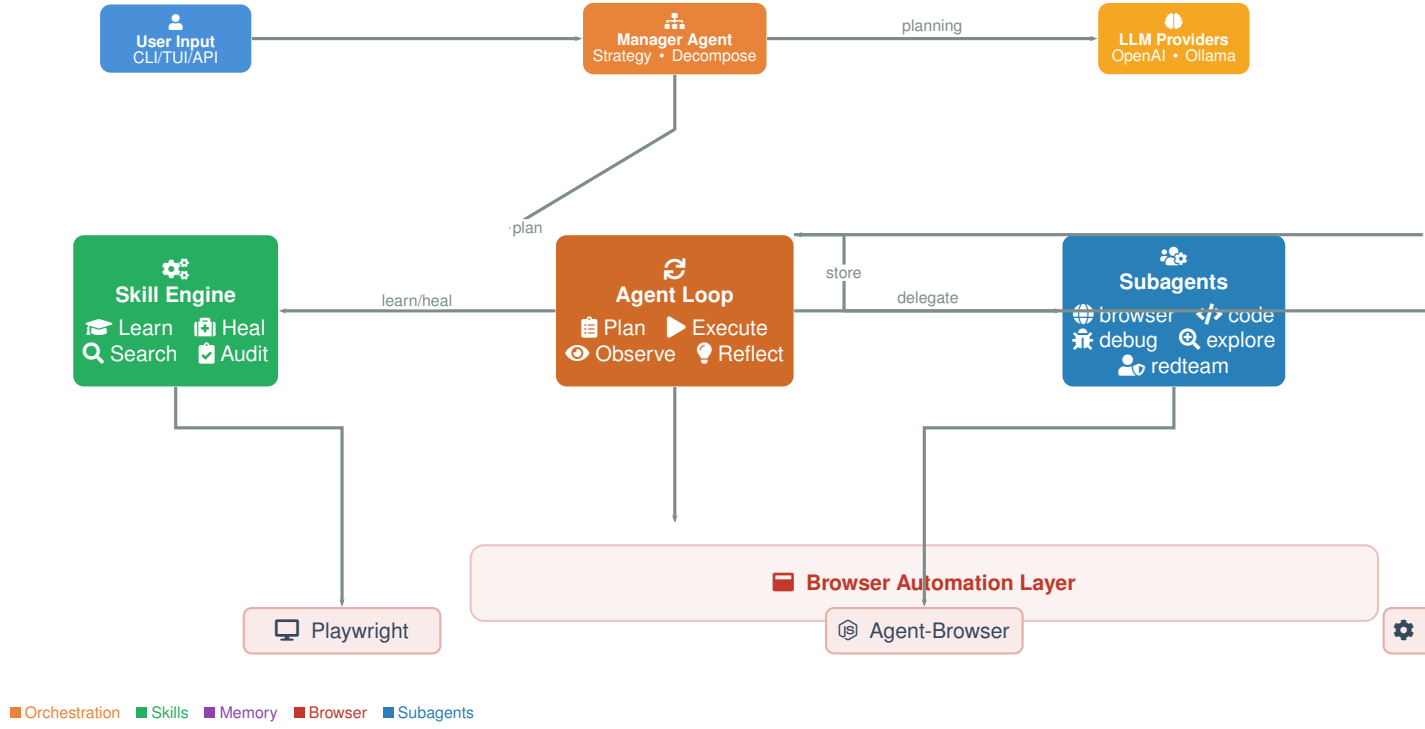


Figure 1: OpenSkynet system architecture.

2.3 Agent Loop

The Agent Loop implements a Plan–Execute–Observe–Reflect cycle inspired by the ReAct paradigm [14]. At each iteration:

1. **Plan:** The manager produces a plan with strategy tags and optional subtask decomposition.
2. **Execute:** The current step is dispatched according to its strategy (direct, skill-based, or delegated to a subagent).
3. **Observe:** The result is classified as success or failure; screenshots and DOM snapshots are captured for downstream analysis.
4. **Reflect:** An LLM-based reflection agent evaluates whether the task is complete, with confidence scoring. If confidence is low or the step failed, a retry with exponential backoff ($2^n + \epsilon$, capped at 10s) is attempted. After exhausting retries, the system falls back through strategies: Use-Skill $\rightarrow$ Direct, Delegate $\rightarrow$ Direct, Decompose $\rightarrow$ Delegate.

A Budget tracker monitors replans (max 3), total retries (max 8), wall-clock time (max 600s), LLM tokens (max 200K), and browser actions (max 100). When any resource is exhausted, the agent terminates gracefully.

2.4 Prompt Optimization

To reduce token consumption, OpenSkynet provides three system prompt tiers:

- **Full mode** (default): Comprehensive system prompt including agent identity, memory context, project context, and task-adaptive add-ons.
- **Flash mode:** Compact system prompt retaining verification rules and error recovery instructions but omitting persona and context.
- **Turbo mode:** Ultra-minimal prompt (3 lines) for simple, well-defined tasks with zero context overhead.

Task-adaptive add-ons are dynamically appended based on keyword matching: extraction tasks receive structured output instructions; form-filling tasks receive field-by-field guidance; search tasks receive query

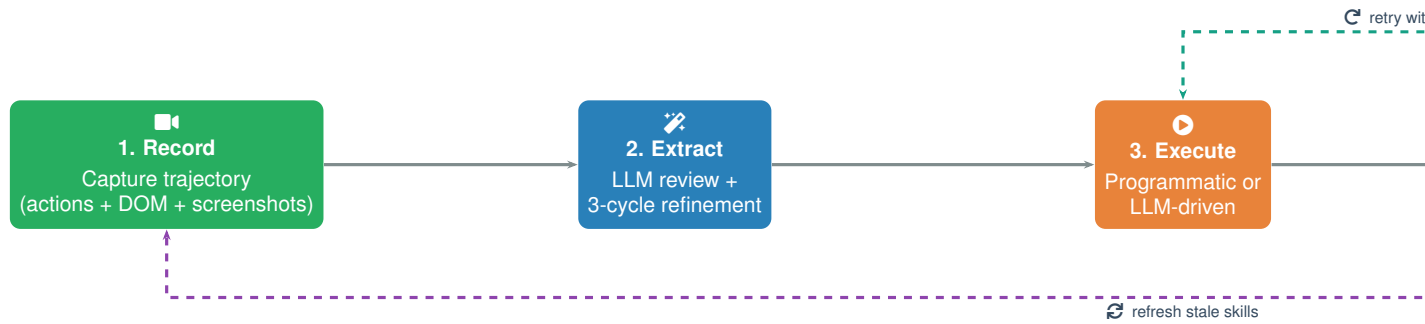


Figure 2: Skill lifecycle: trajectories are recorded, skills are extracted and refined, execution heals on failure, and periodic auditing prunes stale skills.

construction strategies; and navigation tasks receive multi-page state management rules.

2.5 Browser Abstraction Layer

OpenSkynet supports three browser backends through a unified interface:

1. **Playwright / Browser-Use:** The primary backend, providing full Playwright control with stealth mode (CloakBrowser), cookie/localStorage persistence, and 18 discrete browser tools exposed to the LLM (navigate, click, type, scroll, screenshot, extract, etc.).
2. **Agent-Browser Sidecar:** A Node.js process that runs a self-contained AI agent with its own browser control. Communication occurs via JSON-RPC 2.0 over stdin/stdout. Useful when the main agent delegates browsing to an isolated subagent.
3. **OpenBrowser REST:** A Rust-based semantic browser that exposes a REST API with semantic tree introspection. The controller adapter translates REST responses into the same `PageSnapshot` and `ElementInfo` data structures used by the Playwright backend.

All backends share a common `BrowserSession` interface (start/stop/screenshot/save-state/load-state) and a common `BrowserController` interface (18 browser actions). A module-level singleton controller enables transparent backend swapping without modifying tool definitions.

3 Skill Learning and Self-Healing

The skill system is the core differentiator of OpenSkynet. Skills are named, versioned, executable recipes consisting of textual steps, structured browser actions (navigate, click, type, scroll, extract), and rich metadata (verification criteria, retry policy, timeout, allowed tools, input/output signatures). The full lifecycle is shown in Figure 2.

3.1 Skill Storage and Versioning

Skills are stored as dual-format files on disk: `skill.json` (structured, machine-readable) and `SKILL.md` (human-readable Markdown with YAML frontmatter). The Skill Engine supports two storage scopes: a global directory (`~/openskynet/skills/`) and a project-local directory (`.openskynet/skills/`), with project-level skills taking priority. Every update creates a versioned archive in a `history/` subdirectory, enabling rollback to any prior version. A lock file (`skills-lock.json`) tracks installation provenance, version hashes, and update detection for skills installed from the centralized skills hub or GitHub repositories.

3.2 Skill Learning Pipeline

Skill learning occurs automatically after each successful browser task. The `SkillLearnerAgent` orchestrates a multi-stage pipeline:

1. **Pre-check:** If the task was successful and ≥ 2 browser actions were performed, check for similar existing skills using vector similarity search. If a close match exists (cosine similarity ≥ 0.1), skip LLM review and directly decide to patch the existing skill.
2. **LLM Review:** A structured prompt (loaded from a configurable template) asks the LLM to evaluate three criteria: (a) count of actions ≥ 5 , (b) task reusability, and (c) novelty relative to existing skills. At least 2 of 3 must be satisfied to proceed.
3. **Iterative Refinement:** The initial skill draft is refined through up to 3 cycles of LLM critique. Each cycle feeds the current draft back to the model with instructions to critique and improve step specificity, add CSS selectors, identify pitfalls, and suggest verification criteria.
4. **Failure-Aware Context:** Recent failed trajectories (up to 5) are loaded from the trajectory database and included as contextual examples. The LLM is instructed to avoid patterns that led to prior failures.
5. **Similar-Skill Merging:** If a similar skill exists and `should_patch` is set, the new steps are merged into the existing skill rather than creating a duplicate. If no similar skill exists, a new skill is created.
6. **Security Gate:** Before saving, all skill content (name, description, steps) is scanned for prompt injection patterns, credential leaks, and destructive commands.

Skills can also be learned from screen recordings via the `TraceToSkill` module. This module extracts up to 25 key frames from a recorded session, overlays cursor positions on each frame, and sends the multimodal sequence (frames + action log + DOM summaries) to an LLM. A two-phase post-processing step extracts reusable `{{variable}}` placeholders and refines steps with DOM-specific selectors for robustness.

3.3 Self-Healing Execution

When a skill execution fails—whether due to a changed DOM structure, a missing element, an unexpected popup, or a verification failure—the Healer module is invoked automatically:

1. **Evidence Capture:** A screenshot of the browser state at failure and a truncated DOM snapshot (up to 3000 characters) are captured. The screenshot is limited to 5MB to prevent base64 overflow.
2. **Multimodal Diagnosis:** The error context, original skill steps, screenshot (as base64-encoded image), and DOM snapshot are sent to an LLM with a structured healer prompt. The model returns a JSON object containing the identified failing step, root cause, corrected steps, and a confidence level (low/medium/high).
3. **Unfixable Detection:** If the LLM responds with an explicit `error` key, the system marks the skill as unfixable and terminates without patching, preventing cascading failures.
4. **Auto-Patching:** The corrected steps are applied to the skill via the engine’s `patch()` method, which creates a new version and archives the previous one. The confidence and reasoning are logged for audit.
5. **Re-execution:** The healed skill is re-executed. If healing succeeds, the skill’s success rate metric is updated. If it fails again, the skill is marked for manual review.

This creates a *continuous improvement loop*: skills become more robust over time through actual usage, without requiring human intervention to update selectors or steps when web pages change.

3.4 Skill Audit

A periodic `SkillAuditor` reviews installed skills for staleness. Skills unused for 30+ days are flagged; the auditor consults an LLM for additional nuance (e.g., detecting truly broken vs. temporarily unused skills) and recommends actions: archive (set category to “archived”) or delete. The auditor also identifies skills with low user preference scores (from the DPO-inspired preference learner) and suggests pruning.

4 Tiered Memory Architecture

Traditional browser agents operate statelessly across sessions. OpenSkynet introduces a three-tier memory system that progressively promotes knowledge from ephemeral working memory to persistent long-term storage, enabling cross-session improvement.

4.1 Three-Tier Model

The memory hierarchy consists of:

- **Working Memory** (max 20 entries, 4000 characters): Holds observations and lessons from the current task. Populated by the `AutoMemory` module, which scores each observation for importance using a heuristic regex-based classifier (high-value keywords like “always”, “critical”, “bug”, “workaround” receive +0.15; low-value keywords like “maybe”, “test”, “tmp” receive −0.10). Failed actions receive an automatic +0.2 importance boost.
- **Session Memory** (max 100 entries, 20000 characters): Accumulates working memory entries across a session. Entries are evicted based on a combined score: $\text{importance} \times 0.5 + \text{recency} \times 0.3 + \text{access_frequency} \times 0.2$. High-importance entries (≥ 0.7) are promoted to long-term storage.
- **Long-Term Memory**: Persistent dual-file storage (`MEMORY.md` for agent knowledge, `USER.md` for user preferences) using section-sign-delimited entries for human readability. Entries are written atomically via `tempfile + os.replace`. A frozen snapshot is taken at session start to preserve LLM prefix-cache invariance.

4.2 Retrieval Architecture

Memory retrieval employs a multi-layered approach with graceful degradation:

1. **Vector Search**: Entries are embedded and stored in a SQLite-backed vector database with optional `sqlite-vec` extension for native ANN. The embedding provider degrades through three tiers: OpenAI `text-embedding-3-small` (1536d) → local FastEmbed BAAI/bge-small-en-v1.5 (384d) → pure-Python TF-IDF (512d). The TF-IDF provider persists vocabulary and IDF values across sessions, enabling incremental learning.
2. **Keyword Fallback**: When no embedding provider is available, Jaccard-like word overlap scoring over name + description + category + keywords is used.
3. **LLM-Powered Background Review**: Every 10 turns, a background review agent analyzes recent conversation and current memory, suggesting additions, replacements, or removals. Contradiction detection uses negation-pair heuristics (e.g., “always” vs. “never”, “prefer” vs. “avoid”) with word-overlap similarity thresholds.
4. **Context Filtering**: Task-specific context injection uses a multi-factor ranking: text relevance (0.5) + recency (0.25) + access frequency (0.15) + entry-type bonus (0.10–0.15), ensuring only the most pertinent memories consume context window budget.

4.3 Consolidation

When memory limits are reached, the `MemoryConsolidator` applies three strategies in order: (1) merge small entries (< 60 characters) with semicolon separators; (2) summarize old entries by truncating to the first half of sentences; and (3) remove the lowest-scored entries. An optional LLM-based consolidation path summarizes the oldest entries when they account for at least 50% of the needed space, avoiding wasted LLM calls on trivial memory pressure.

4.4 Security

All memory writes pass through a content security scanner that detects prompt injection attempts, credential leaks (`api_key`, `token`, `secret`), backdoor commands (`authorized_keys`), destructive operations (`rm -rf /`, `DROP TABLE`), and invisible Unicode characters (zero-width, bidi overrides, format characters). A streaming context scrubber removes `<memory-context>` tags from LLM output to prevent internal memory state from leaking to users.

5 Multi-Agent Subagent System

OpenSkynet includes a multi-agent framework for decomposing complex tasks across specialized subagents, each with restricted permissions and optionally isolated execution environments.

5.1 Subagent Definition

Subagents are defined as Markdown files with YAML frontmatter, making them easy to create, share, and customize. Each subagent specification includes: a name, description, mode (defaulting to “subagent”), an optional model override, a permission map (tool → allow/ask/deny), a maximum iteration limit, and a system prompt body. OpenSkynet ships with 7 built-in subagents:

- **Browser** (max 8 iterations): Web browsing, form filling, data extraction. Denied: terminal, file write.
- **Code** (max 30 iterations): Full-featured coding agent with a 5-stage workflow (Explore → Plan → Execute → Verify → Summarize). Denied: browser.
- **Debug** (max 5 iterations): Diagnostic agent for analyzing failures. Read-only access to files and terminal.
- **Explore** (max 3 iterations): Rapid survey agent. Max 5 bullet points in summary. No file writes.
- **Integrate** (max 50 iterations): Merge feature branches and resolve conflicts. Full terminal access.
- **Review** (max 3 iterations): Critique and verify existing work. Rates as PASS/NEEDS_FIX/REJECT.
- **RedTeam** (max 10 iterations): Adversarial test engineer. Creates edge-case tests (null, empty, large, Unicode, concurrent).

User-defined subagents can be added to `~/ .openskynet/agents/` and may override built-in agents by name.

5.2 Permission-Based Isolation

Each subagent inherits a permission map that filters the global tool registry before execution. Permissions are resolved in priority order: `deny` (by skill name) → `deny` (by source) → `always_allow_source` → `always_allow_skill` → `allow_once` → `skip` → `none`. When a tool’s permission is “ask”, an async approval callback is invoked, enabling human-in-the-loop approval for high-risk actions.

This enforces the principle of least privilege: a browser subagent cannot execute terminal commands; a code subagent cannot navigate web pages; and a review agent cannot modify files. Risk assessment (`assess_risk`) classifies tool calls as low, medium, or high risk based on command patterns (safe command whitelist, risky URL patterns, destructive tool identifiers), enabling additional gating for dangerous operations.

5.3 Parallel Execution

When the Manager Agent decomposes a task into multiple independent subtasks (Decompose strategy), the `SubagentFactory` spawns up to 3 subagents concurrently using `asyncio.Semaphore` for concurrency control. Each subagent can optionally receive an isolated browser context (temporary directory, headless, stealth mode) to prevent session interference. Results are gathered in order and merged.

A shared scratchpad—a versioned key-value store—enables subagents to communicate context across execution boundaries. Parent context (task description, recent errors, recent observations) is injected into each

subagent’s system prompt via `<parent_task>`, `<parent_errors>`, and `<parent_observations>` tags.

5.4 Nesting Control

To prevent infinite recursion, the `SubagentFactory` tracks nesting depth and refuses to spawn beyond a configurable maximum depth. This is essential when a code subagent delegates a subtask to another code subagent, which could otherwise cascade indefinitely.

6 The Evil Skill Generalization Benchmark

Evaluating self-healing and skill generalization requires testing conditions where skills *will* break, and measuring whether the system can recover. Standard benchmarks (WebArena, Mind2Web) evaluate execution on canonical web pages but do not systematically stress-test the failure-and-recovery path. We propose the *Evil Skill Generalization Benchmark*, a deliberately adversarial test suite designed to fill this gap.

6.1 Design Principles

The benchmark is built around three principles: (1) skills should break on parameter variants to test healing; (2) pages should be intentionally obfuscated to test robustness; and (3) the same page template should yield different experiences across variants to test generalization.

6.2 Task Structure

The benchmark comprises 60 tasks across 20 web application templates, each with 3 parameter variants. Templates span four domains: **finance** (Yahoo Finance, Google Finance, CoinMarketCap, SEC EDGAR), **e-commerce** (Amazon, Newegg, Zillow, Craigslist, eBay), **knowledge** (Wikipedia, IMDb, Goodreads, Crunchbase), and **utilities** (weather, currency, flights, packages, WHOIS, GitHub, government databases). Each template specifies expected extraction keys and a golden-path navigation sequence.

6.3 Obfuscation Layers

Each template page implements 7 anti-LLM obfuscation techniques:

1. **Dynamic injection:** Navigation buttons injected by JavaScript after a deliberate delay, absent from the initial DOM.
2. **Visibility attacks:** Interactive elements rendered with `opacity: 0.005` or `visibility: hidden`, requiring DOM-level inspection.
3. **Multi-layer consent:** Stacked consent walls (ToS, checkbox, CAPTCHA, accordion), each layer parameterized per variant.
4. **Ghost decoys:** 10 near-identical transparent buttons; 9 reset progress on click, only 1 advances the task.
5. **CAPTCHA challenges:** Codes rendered on canvas elements, requiring vision or DOM-level verification.
6. **Conditional DOM injection:** Content dynamically created after accordion expansion, not present in static HTML.
7. **Variant-specific identifiers:** HTML element IDs incorporate variant parameters, preventing hardcoded selectors.

6.4 Evaluation Protocol

Each task is evaluated in two modes: **without-skill** (baseline, the agent reasons from scratch) and **with-skill** (a skill recorded on one variant is replayed on the remaining two). Metrics include binary success/failure, partial-match scoring (exact: 1.0, partial: 0.5, normalized per-key), steps taken, wall-clock time, and token consumption. A task is considered successful if aggregate score ≥ 0.67 .

6.5 Implementation

A mock web server serves the 20 obfuscated templates locally, enabling controlled, reproducible evaluation without dependency on live websites. The benchmark infrastructure includes a canonical skill recorder, a batch runner with isolated browser profiles, a results evaluation script, and a report generator. Full-scale evaluation is ongoing and will be reported in future work.

7 Related Work

7.1 LLM-Powered Browser Agents

The field of LLM-based web agents has grown rapidly. Browser-Use [1] provides a Playwright-based agent framework that integrates vision-language models for web interaction. SeeAct [16] pioneered the use of GPT-4V for integrated visual understanding and action grounding, achieving 51.1% on live websites with manual grounding. WebRL [10] introduced self-evolving online curriculum RL, improving Llama-3.1-8B from 4.8% to 42.4% on WebArena-Lite. CUA-Gym [12] demonstrated scalable RL with verifiable rewards, reaching 72.6% on OSWorld-Verified. OpenSkynet differs from these systems in its focus on *post-execution improvement*: rather than improving single-task performance through training, it improves cross-task robustness through self-healing and skill reuse at inference time.

7.2 Skill Reuse and Memory

Several recent works address skill reuse and memory in agent systems. MUSE-Autoskill [3] introduces a skill-centric framework with a unified skill lifecycle (creation, memory, management, evaluation, refinement). MEMENTO [2] treats the web as a learning signal, with dual-channel memory (declarative knowledge + procedural strategies). ExpeL [15] extracts insights from successful and failed trajectories via LLM reflection. OpenSkynet’s skill learning pipeline is inspired by these approaches but adds iterative refinement with failure-aware context and similar-skill merging. The three-tier memory model with heuristic importance scoring and background consolidation is novel in the browser automation space.

7.3 Self-Healing and Error Recovery

Self-healing has been explored in GUI automation (RoSUE [5] repairs UI test scripts) and robotic task execution (Robo-Cortex [4] maintains a Navigation Heuristic Library). CRITIC [7] validates agent outputs with external tools before committing. OpenSkynet’s healer extends these ideas to browser automation by combining multimodal evidence (screenshot + DOM) with LLM reasoning to produce targeted step-level patches, versioning each fix for auditability.

7.4 Agent Decomposition and Planning

Plan-and-execute architectures [14] decompose tasks into subtasks for focused execution. ReAct [14] interleaves reasoning and action in a single loop. Reflexion [11] adds verbal reinforcement learning through stored reflections. MiRA [13] uses subgoal-driven planning with milestone-based rewards. OpenSkynet’s Manager Agent combines task classification, strategy selection, and beam-width decomposition; the subagent system extends these ideas with permission-based isolation and controlled parallel execution.

7.5 Benchmarks for Browser Agents

WebArena [17] and VisualWebArena provide reproducible environments for evaluating web agents. Mind2Web [6] tests generalization to unseen websites. WebVoyager [9] evaluates on live websites. The Evil Skill Generalization Benchmark proposed in this work is complementary: it focuses on *adversarial robustness* and *skill generalization under parameter variation*, dimensions not systematically tested by existing benchmarks.

8 Discussion and Future Work

OpenSkynet demonstrates that self-healing, skill learning, and tiered memory can be integrated into a unified browser automation framework. The system’s architecture addresses three persistent limitations of LLM-powered agents: execution brittleness, lack of cross-session learning, and single-agent context window constraints.

8.1 Limitations

Several limitations motivate future work. First, the reliance on LLMs for both execution and healing creates a circular dependency: the same model that failed to execute the original task is asked to diagnose and repair the failure. Exploring external validation mechanisms—such as execution-based verifiers or programmatic constraint checkers—could reduce this dependency. Second, the skill learning pipeline currently generates text-based steps; incorporating structured execution traces with element selectors, wait conditions, and error handlers could improve robustness. Third, the memory system’s importance scoring is heuristic-based; adopting learned importance models (e.g., fine-tuned classifiers or RL-trained critics) could improve promotion accuracy.

8.2 Future Directions

- **Full-scale benchmark evaluation.** Running the Evil Skill Generalization Benchmark across all 60 tasks with multiple LLM providers will produce quantitative results on self-healing efficacy and skill generalization rates.
- **Standard benchmark integration.** Evaluating OpenSkynet on WebArena and VisualWebArena will provide apples-to-apples comparison with existing browser agent systems.
- **Ablation studies.** Systematic ablation of individual components (healing on/off, skill learning on/off, memory tier on/off) will isolate the contribution of each mechanism.
- **RL fine-tuning.** Combining the framework’s trajectory database with reinforcement learning (inspired by WebRL’s self-evolving curriculum) could train specialized models that are inherently more robust to page changes.
- **Skill marketplace.** OpenSkynet’s skills hub currently supports 470+ community skills. Formalizing skill sharing with trust levels, verification protocols, and reputation systems could create a collaborative ecosystem for browser automation skills.
- **Cross-modal skill transfer.** Extending the Trace-to-Skill module to accept video demonstrations (rather than just browser recordings) would enable users to “teach” the system by recording their screen, further lowering the automation barrier.

8.3 Conclusion

We presented OpenSkynet, a framework that combines self-healing skill execution, automated skill learning, tiered memory, and multi-agent orchestration for persistent browser automation. The system’s architecture is designed to improve over time: skills become more robust through healing, memory accumulates cross-session knowledge, and subagents enable scalable task decomposition. The proposed Evil Skill Generalization Benchmark provides a principled testbed for evaluating self-healing and generalization in adversarial web environments. We release OpenSkynet as open-source software to facilitate further research and practical adoption in the browser automation community.

References

- [1] Browser-Use Contributors. Browser-use: Ai-powered browser automation. 2024. <https://github.com/browser-use/browser-use>.

- [2] MEMENTO Contributors. Memento: Treating the web as a learning signal for browser agents. *arXiv preprint arXiv:2605.29795*, 2026.
- [3] MUSE-Autoskill Contributors. Muse-autoskill: Skill-centric agent framework with unified skill lifecycle. *arXiv preprint arXiv:2605.27366*, 2026.
- [4] Robo-Cortex Contributors. Robo-cortex: Self-evolving framework with dual-grain cognitive memory. *arXiv preprint arXiv:2605.18729*, 2026.
- [5] RoSUE Contributors. Rosue: Robust self-healing ui test automation. 2025.
- [6] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *arXiv preprint arXiv:2306.06070*, 2023.
- [7] Zhibin Gou et al. Critic: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint arXiv:2305.11738*, 2023.
- [8] Izzeddin Gur et al. Webagent: A scalable agent for complex web tasks. *arXiv preprint arXiv:2312.11805*, 2023.
- [9] Hongliang He et al. Webvoyager: Building an end-to-end interactive web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*, 2024.
- [10] Zehan Qi, Xiao Liu, et al. Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning. *arXiv preprint arXiv:2411.02337*, 2024.
- [11] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.
- [12] Bowen Wang et al. Cua-gym: Scalable rlvr pipeline for computer-using agents. *arXiv preprint arXiv:2605.25624*, 2026.
- [13] Yuxuan Wang et al. Mira: Subgoal-driven planning with milestone-based rewards for web agents. *arXiv preprint arXiv:2603.19685*, 2026.
- [14] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
- [15] Andrew Zhao, Daniel Huang, Quentin Xu, Zander Lin, Nelson Liu, and Hao Peng. Expel: Llm agents are experiential learners. In *AAAI*, 2024.
- [16] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Seeact: Gpt-4v(ision) is a generalist web agent, if grounded. *arXiv preprint arXiv:2401.01614*, 2024.
- [17] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.